POLICY GRADIENT LAB

REINFORCE · value baselines · GAE

A reproducible study of variance reduction and sample efficiency

4 completed runs / 4 environments / primary training seed: 1

CartPole-v0

Best evaluation-batch return: 200.0

Configuration: cartpole_na · training seed 1

HalfCheetah-v4

No completed measurements yet

LunarLander-v2

No completed measurements yet

InvertedPendulum-v4

No completed measurements yet

Peaks are descriptive single-batch observations, not confidence estimates.
Final returns, trailing means, raw logs and limitations appear inside.

Based on Berkeley CS285 Spring 2026 Homework 2
Generated directly from versioned experiment CSVs.

Measured evaluation returns (seed 1)

Experiment | peak | final | last 10 mean | first target steps

cartpole_na | 200.0 | 200.0 | 200.0 | 21151
cartpole_rtg_na | 200.0 | 200.0 | 165.5 | 18004
cartpole_rtg | 200.0 | 200.0 | 186.7 | 41159
cartpole | 200.0 | 122.0 | 126.3 | 20711

1. CartPole: returns, normalization and batch size

Small batch (1,000). Last-ten mean evaluation returns: trajectory 126.3, reward-to-go 186.7, normalized trajectory 200.0, normalized reward-to-go 165.5. Without normalization, reward-to-go performed better by this endpoint metric.

Normalization changed the trajectory last-ten evaluation mean by +73.7 points.

Normalization changed the reward-to-go last-ten evaluation mean by -21.2 points.

Reward-to-go is generally preferred because an action cannot influence past rewards. Removing those past terms preserves the expected score-function gradient while usually reducing its variance. Relative discounting weights future rewards from each decision.

Normalization rescales the update and subtracts a batch mean; it is a useful heuristic but introduces finite-batch bias. Compare normalized and unnormalized rows above rather than assuming it always helps. Batch size trades gradient noise against interaction cost: 100 iterations at 4,000 steps use roughly four times the data of 1,000-step batches. The charts compare actual training steps, not iteration counts. CartPole plots show training-batch average return, as in the assignment; the comparison table uses evaluation return.

2. HalfCheetah: learned baseline

The one-gradient-step ablation keeps the baseline learning rate at 0.01 and reduces value updates from five to one. This slows fitting to the current batch; policy outcomes above show whether that change helped this seed. Value targets change with the policy, so training MSE need not decrease monotonically, and lower training MSE alone does not prove better control. The baseline uses raw targets.

3. LunarLander: generalized advantage estimation

λ=0 uses one-step TD residuals, reducing variance while depending heavily on value accuracy. λ=1 telescopes to Monte Carlo reward-to-go minus the baseline under the zero-tail boundary convention, reducing bootstrap bias while increasing variance. Intermediate λ mixes horizons. The observed ranking above and curves below reflect this seed, not a universal monotonic relationship. All five runs retain the PDF’s fixed batch size, learning rate, architecture, discount, and iteration budget.

The official starter uses standard LunarLander-v2 without an additional action-noise wrapper; this protocol is preserved. See PROVENANCE.md.

4. InvertedPendulum: sample efficiency

No completed run has reached the 1,000-return target yet.

Tuning reduces batches from 5,000 to 500 or 1,000 transitions to allow more frequent policy updates per interaction budget, and combines reward-to-go, normalization, a value baseline, γ=0.99 and GAE. The candidates also vary learning rate and λ. These coupled changes are a configuration search, not a controlled causal attribution of gains to any one hyperparameter. All attempts are retained.

Reproduction and limitations

Exact command lines are in commands.sh; per-run flags and source revision are beside

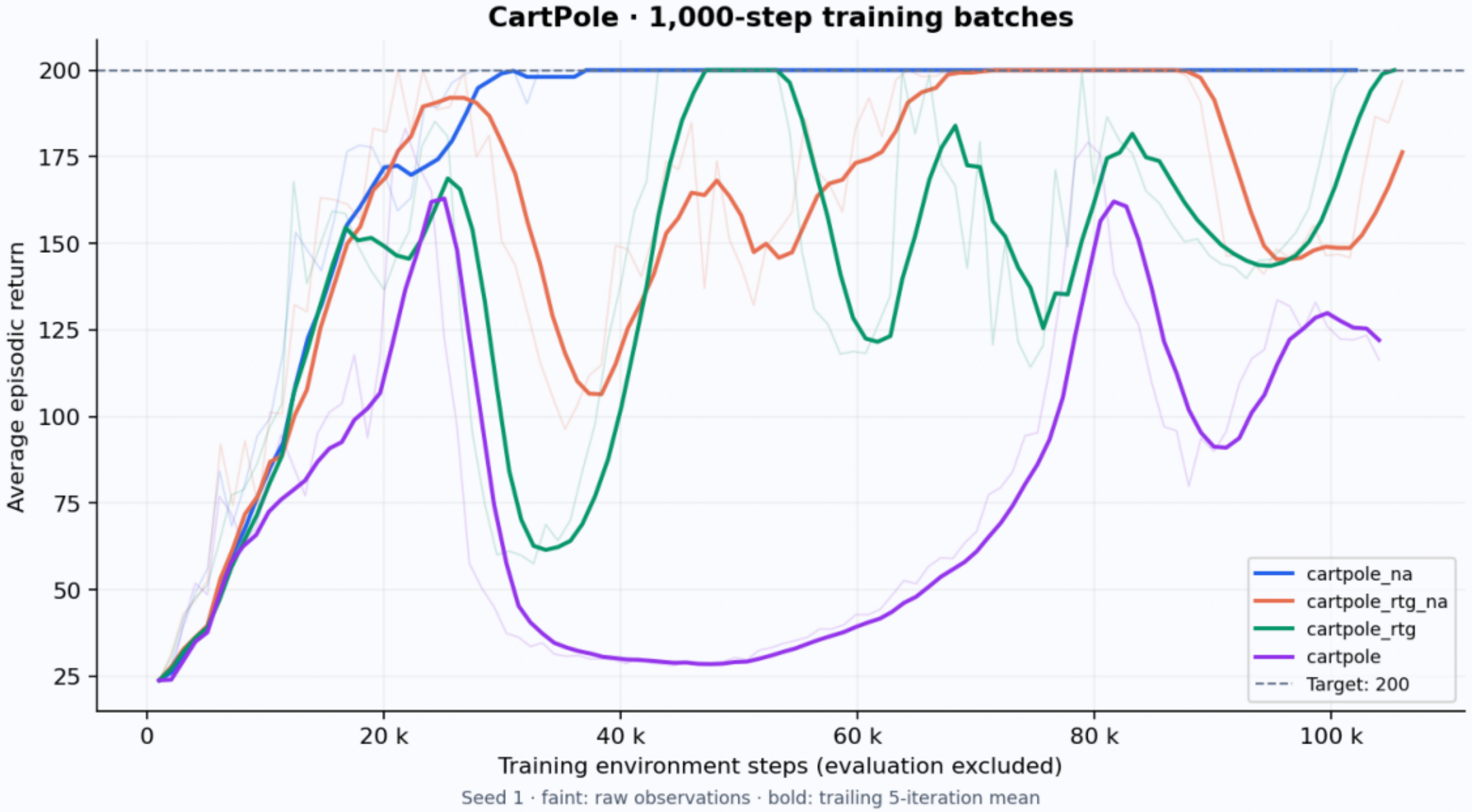
each CSV. The eight CartPole, three HalfCheetah, five LunarLander, and four InvertedPendulum configurations are generated by scripts/experiments.py.

Single-seed comparisons and peak selection can be optimistic. A larger study should repeat every setting over independent seeds and evaluate saved policies over many held-out episodes. Training steps exclude evaluation interactions as required by the assignment. Time-limit tails are zero, not bootstrapped. Videos show individual rollouts of final checkpoints and do not establish average performance.

Fresh-episode evaluation of selected final checkpoints

Configurations are selected from the benchmark comparisons. Each frozen final policy is then evaluated stochastically on 30 new, individually seeded episodes. These bootstrap intervals measure episode variation for one trained policy, not training-seed uncertainty.

Selected policy	Episodes	Mean return	Standard deviation	Bootstrap 95% CI
[cartpole_na](evaluation/cartpole.json)	30	200.0	0.0	[200.0, 200.0]


CartPole · 1,000-step training batches
Average episodic return
200
175
150
125
100
75
50
25
0
20 k
40 k
60 k
80 k
100 k
Training environment steps (evaluation excluded)
cartpole_na
cartpole_rtg_na
cartpole_rtg
cartpole
Target: 200
Seed 1 · faint: raw observations · bold: trailing 5-iteration mean

Exact experiment commands

Generated exact benchmark commands; run from the repository root.
export OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 --exp_name cartpole --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg --exp_name cartpole_rtg --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -na --exp_name cartpole_na --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na --exp_name cartpole_rtg_na
--no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 --exp_name cartpole_lb --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg --exp_name cartpole_lb_rtg
--no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -na --exp_name cartpole_lb_na --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na --exp_name cartpole_lb_rtg_na
--no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr
0.01 --exp_name cheetah --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr
0.01 --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr
0.01 --use_baseline -blr 0.01 -bgs 1 --exp_name cheetah_baseline_bgs1 --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb
2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0 --exp_name
lunar_lander_lambda0 --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb
2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.95 --exp_name
lunar_lander_lambda0.95 --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb
2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.98 --exp_name
lunar_lander_lambda0.98 --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb
2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.99 --exp_name
lunar_lander_lambda0.99 --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb
2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 1 --exp_name
lunar_lander_lambda1 --no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 5000 -eb 1000 --exp_name pendulum
--no_gpu --rollout_backend numpy
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 1000 -eb 1000 -rtg -na --discount
0.99 -lr 0.01 --use_baseline --gae_lambda 0.95 -bgs 10 --exp_name pendulum_tuned --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 500 -eb 1000 -rtg -na --discount
0.99 -lr 0.005 --use_baseline --gae_lambda 0.98 -bgs 10 --exp_name pendulum_tuned_small --no_gpu
--rollout_backend numpy
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 1000 -eb 1000 -rtg -na --discount
0.99 -lr 0.02 --use_baseline --gae_lambda 0.95 -bgs 10 --exp_name pendulum_tuned_fast --no_gpu
--rollout_backend numpy